

Week 01 Revision Questions

1. Do I *need* to set up Rust on my own computer, or can I just use VLAB / CSE?

How could I go about setting up Rust on my own computer?

ANSWER:

Do I *need* to set up Rust on my own computer, or can I just use VLAB / CSE?

You will be able to use VLAB / CSE for this course. Setting up Rust on your own computer is suggested, but not required.

How could I go about setting up Rust on my own computer?

We have written a guide and an instructional video in the Home Computing section on the [here](#).

2. Let's write our first Rust program, hello world!

First we'll write it as a `main.rs` file, and use the `rustc` command to compile it.

Then, we will modify our program to be a cargo project instead.

What are the differences in the two approaches? When should we use each approach in COMP6991?

ANSWER:

[COMP6991] Hello World, rustc vs cargo



`main.rs` is a freestanding Rust file that must be compiled with the `rustc` command. It is not a cargo project, which means that all dependency management must be done manually.

Cargo is a Rust package manager. It is a tool that can be used to manage dependencies, build Rust projects, and more. In the real world it is incredibly rare to find Rust projects not making use of Cargo, but we will use both approaches throughout COMP6991.

3. Translate the following C program into Rust:

```
#include <stdio.h>

int main(void) {
    int my_integer = 42;

    if (my_integer > 0) {
        printf("%d is positive\n", my_integer);
    } else if (my_integer < 0) {
        printf("%d is negative\n", my_integer);
    } else {
        printf("%d is zero\n", my_integer);
    }

    return 0;
}
```

ANSWER:

[COMP6991] If, else if, else in Rust



```
fn main() {  
    let my_integer = 42;  
  
    if my_integer > 0 {  
        println!("{my_integer} is positive");  
    } else if my_integer < 0 {  
        println!("{my_integer} is negative");  
    } else {  
        println!("{my_integer} is zero");  
    }  
}
```

4. Translate the following C program into Rust:

```
#include <stdio.h>  
  
int main(void) {  
    int count = 0;  
  
    while (count < 10) {  
        printf("%d\n", count);  
        count++;  
    }  
  
    return 0;  
}
```

ANSWER:

[COMP6991] Looping in Rust



```
fn main() {  
    let mut count = 0;  
  
    while count < 10 {  
        println!("{count}");  
        count += 1;  
    }  
}
```

5. Translate the following C program into Rust:

```
#include <stdio.h>

int main(void) {
    for (int count = 0; count < 10; count++) {
        printf("%d\n", count);
    }
}
```

Is it possible to translate this approach directly to Rust? Why / why not?

ANSWER:

[COMP6991] Looping in Rust



Rust does not have support for C-style for loops. Instead, Rust allows iterator-based for loops, as seen in languages such as Python:

```
fn main() {
    for count in 0..10 {
        println!("{count}");
    }
}
```

6. Translate the following C program into Rust:

```
#include <stdio.h>

void my_function(int *x);

int main(void) {
    int x = 42;

    my_function(NULL);
    my_function(&x);

    return 0;
}

void my_function(int *x) {
    if (x == NULL) {
        printf("no value for x\n");
    } else {
        printf("x has value %d\n", *x);
    }
}
```

Is this program possible to write in Rust? Discuss whether this is an advantage or disadvantage of Rust.

ANSWER:

[COMP6991] Optional values in Rust



Rust does not have support for null references. Instead we model a value that may or may not exist with the `Option` type.

```
fn main() {
    let x = 42;

    my_function(None);
    my_function(Some(x));
}

fn my_function(maybe_x: Option<i32>) {
    match maybe_x {
        Some(x) => println!("x has value {x}"),
        None    => println!("no value for x"),
    }
}
```

This is, in almost all situations, an overwhelming advantage of Rust. See the inventor of the null pointer: Tony Hoare's [Null References: The Billion Dollar Mistake](#).

COMP6991 23T1: Solving Modern Programming Problems with Rust is brought to you by
the [School of Computer Science and Engineering](#)
at the [University of New South Wales](#), Sydney.

For all enquiries, please email the class account at cs6991@cse.unsw.edu.au

CRICOS Provider 00098G

[Login as tutor](#)